

# SafePay V1 — Updated Full Implementation Plan

---

## 1. Development Charter

---

This plan replaces the earlier implementation plan as the main reference while preserving its approved business logic, APIs, security controls, database integrity rules, and feature scope.

### Approved V1 behaviour

| Payment amount          | Risk tier | Protection action                                    |
|-------------------------|-----------|------------------------------------------------------|
| ₹0.01–₹5,000.00         | LOW       | Release and settle immediately                       |
| ₹5,000.01–₹25,000.00    | MEDIUM    | 10-second customer protection window                 |
| ₹25,000.01–₹2,00,000.00 | HIGH      | 60-second customer protection window with warning    |
| Above ₹2,00,000.00      | VERY_HIGH | Hard hold → OTP verification → Risk Officer decision |

- The server is authoritative for timers and transaction state.
- MEDIUM and HIGH payments may be cancelled before release.
- VERY\_HIGH payments never expire automatically.
- An approved VERY\_HIGH payment is released and settled.
- A rejected payment becomes CANCELLED.
- A Risk Officer may request fresh verification, returning it to VERIFICATION\_REQUIRED.
- Device, location, velocity, beneficiary age, amount anomaly, and

weighted scoring are excluded from V1.

- `riskScore` remains nullable in V1.
- The evaluated risk tier, policy/model version, matching amount band, explanation, and timestamp must still be recorded.
- SafePay remains a simulated pre-settlement system and must not claim to reverse completed external bank-rail settlement.

## Development method

The work is divided into five gated subplans:

1. Prerequisites, project setup, and database design
2. Backend and REST API development
3. Complete backend verification and API testing
4. Oracle JET PWA frontend development
5. Deployment and server configuration

For every implementation unit:

1. Explain the concept and its SafePay purpose.
2. Give the exact folder and filename the user must manually create.
3. Provide the complete content to paste.
4. Explain important annotations and logic.
5. Compile or test the unit.
6. Test the corresponding API where applicable.
7. Resolve current errors.
8. Record genuinely future-dependent work in a carry-forward register.
9. Move forward only after the phase exit gate passes.

The agent must not directly create or modify implementation files. All project changes are performed manually by the user using the supplied instructions and content.

---

## 2. Target Project Structure

---

The user-provided layered structure remains the foundation. Missing packages and classes are added only when their corresponding implementation phase begins.

```
SafePay/
├── AGENTS.md
├── SafePay_PRD.md
├── SafePay_PRD.pdf
├── Documentation/
├──
├── backend/
│   ├── pom.xml
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── .mvn/
│   └── src/
│       ├── main/
│       │   ├── java/com/ofss/
│       │   │   ├── SafePayApplication.java
│       │   │   └── beans/
│       │   │       ├── User.java
│       │   │       ├── Role.java
│       │   │       ├── RoleName.java
│       │   │       ├── Account.java
│       │   │       ├── Beneficiary.java
│       │   │       ├── TransactionDb.java
│       │   │       ├── AuditLog.java
│       │   │       ├── RiskTier.java
│       │   │       ├── TransactionState.java
│       │   │       ├── RiskPolicy.java
│       │   │       ├── RiskPolicyBand.java
│       │   │       ├── ProtectionPolicy.java
│       │   │       ├── TransactionRiskFactor.java
│       │   │       ├── OtpChallenge.java
│       │   │       ├── RiskReview.java
│       │   │       ├── RiskReviewStatus.java
│       │   │       ├── IdempotencyRecord.java
│       │   │       └── LedgerEntry.java
```

```
|— LedgerEntryType.java
|— TransactionExceptionLog.java
└— Notification.java
```

```
|— controller/
```

```
|— AuthController.java
|— AccountController.java
|— BeneficiaryController.java
|— TransactionController.java
|— VerificationController.java
|— RiskReviewController.java
|— AuditLogController.java
|— NotificationController.java
|— SystemHealthController.java
```

```
|— repository/
```

```
|— UserDao.java
|— RoleDao.java
|— AccountDao.java
|— BeneficiaryDao.java
|— TransactionDao.java
|— AuditLogDao.java
|— RiskPolicyDao.java
|— RiskPolicyBandDao.java
|— ProtectionPolicyDao.java
|— TransactionRiskFactorDao.java
|— OtpChallengeDao.java
|— RiskReviewDao.java
|— IdempotencyRecordDao.java
|— LedgerEntryDao.java
|— TransactionExceptionDao.java
|— NotificationDao.java
```

```
|— services/
```

```
|— UserService.java
|— UserServiceImpl.java
|— AuthService.java
|— AuthServiceImpl.java
|— AccountService.java
|— AccountServiceImpl.java
```

- └─ BeneficiaryService.java
- └─ BeneficiaryServiceImpl.java
- └─ TransactionService.java
- └─ TransactionServiceImpl.java
- └─ AmountRiskEngine.java
- └─ RiskPolicyService.java
- └─ RiskPolicyServiceImpl.java
- └─ TransactionStateService.java
- └─ TransactionStateServiceImpl.java
- └─ OtpService.java
- └─ OtpServiceImpl.java
- └─ RiskReviewService.java
- └─ RiskReviewServiceImpl.java
- └─ LedgerService.java
- └─ LedgerServiceImpl.java
- └─ IdempotencyService.java
- └─ IdempotencyServiceImpl.java
- └─ AuditLogService.java
- └─ AuditLogServiceImpl.java
- └─ NotificationService.java
- └─ NotificationServiceImpl.java

- └─ dto/

- └─ auth/
- └─ account/
- └─ beneficiary/
- └─ transaction/
- └─ verification/
- └─ review/
- └─ audit/
- └─ notification/

- └─ security/

- └─ JwtService.java
- └─ JwtAuthenticationFilter.java
- └─ CustomUserDetailsService.java
- └─ RestAuthenticationEntryPoint.java
- └─ RestAccessDeniedHandler.java

- └─ config/

```
|
|
|
|
|   ├── SecurityConfig.java
|   ├── WebSocketConfig.java
|   ├── CorsConfig.java
|   └── JacksonConfig.java
|
|   ├── scheduler/
|   │   └── ProtectedTransactionScheduler.java
|
|   ├── common/
|   │   ├── CorrelationIdFilter.java
|   │   ├── MoneyUtility.java
|   │   └── ClockConfiguration.java
|
|   └── excp/
|       ├── ResourceNotFoundExcp.java
|       ├── InvalidStateTransitionException.java
|       ├── BusinessException.java
|       ├── DuplicateRequestException.java
|       ├── UnauthorizedOperationException.java
|       ├── VerificationFailedException.java
|       └── GlobalExceptionHandler.java
|
|   └── resources/
|       ├── application.yml
|       ├── application-local.yml
|       ├── application-test.yml
|       ├── application-prod.yml
|       └── db/migration/
|           ├── V1__security_and_users.sql
|           ├── V2__accounts_and_beneficiaries.sql
|           ├── V3__risk_policy.sql
|           ├── V4__transactions.sql
|           ├── V5__verification_and_review.sql
|           ├── V6__ledger_and_exceptions.sql
|           ├── V7__audit_and_notifications.sql
|           ├── V8__indexes_and_constraints.sql
|           └── V9__prototype_seed_data.sql
|
|   └── test/
|       ├── java/com/ofss/
|       └── resources/
```

```
|
└─ frontend/
    ├── oraclejetconfig.json
    ├── package.json
    ├── tsconfig.json
    └── src/
        ├── index.html
        ├── manifest.json
        ├── service-worker.js
        ├── css/
        ├── js/
        │   ├── main.ts
        │   ├── appController.ts
        │   ├── router/
        │   ├── services/
        │   ├── viewModels/
        │   └── views/
        └── images/
└─ web/
```

`TransactionDb` is retained exactly as requested and represents the payment entity. API payloads must use DTOs rather than exposing this persistence entity.

---

## 3. Subplan 1 — Prerequisites, Project Setup, and Oracle Database

---

### Phase 1.1 — Requirement and contract freeze

Validate before implementation:

- Amount-band boundary behaviour.
- State-transition table.
- Cancellation eligibility.
- VERY\_HIGH OTP and Risk Officer workflow.

- Settlement and double-entry rules.
- Roles and endpoint permissions.
- REST resource names and response conventions.
- Server-authoritative time handling.
- Idempotency rules.
- V1 exclusions and V2 extension points.

Deliverable: an approved requirements matrix and state-transition matrix.

Exit gate: no unresolved conflict between the PRD, supplied diagrams, AGENTS.md, and the latest user-approved requirements.

## Phase 1.2 — Basic installation checks

Check the system and provide corrective installation steps for:

- Java 21 LTS runtime and compiler.
- `JAVA_HOME` and terminal/Eclipse Java selection.
- Maven 3.9.x or Maven Wrapper.
- Oracle Database Free and the `FREEPDB1` service.
- Oracle SQL Developer and a tested local connection.
- Git availability, if version control is introduced.
- Node.js presence only; Node/OJET installation is deferred to Subplan 4.

Expected SQL Developer connection:

- Host: `localhost`
- Port: `1521`
- Service: `FREEPDB1`
- User: dedicated SafePay schema user
- Connection type: Basic

Exit gate: `java -version`, `javac -version`, Maven, Oracle connection, and a simple database query all succeed with compatible versions.

## Phase 1.3 — Backend project bootstrap

Manually generate a Maven Spring Boot 3.x project using Java 21.

Initial dependencies:

- Spring Web
- Spring Data JPA
- Spring Validation
- Spring Security
- Spring WebSocket
- Oracle JDBC driver
- Flyway Oracle support
- JWT library
- Spring Boot Actuator
- JUnit 5
- Mockito
- Spring Security Test
- Testcontainers Oracle-compatible integration support where feasible

Packages are introduced gradually. Empty speculative classes are not created in bulk.

Exit gate: the base application compiles and starts against its selected profile.

## **Phase 1.4 — Oracle security model**

Create distinct database identities:

- `SAFEPAY_OWNER`: owns tables, sequences, indexes, constraints, and migrations.
- `SAFEPAY_APP`: restricted runtime account.
- Separate integration-test schema or disposable test database.

Rules:

- Never run the application as `SYSTEM`.
- Grant only required DML and sequence access to the runtime account.
- Store database credentials outside source control.

- Use Hibernate `ddl-auto=validate`; schema creation belongs to Flyway.
- Use dedicated Oracle sequences for every major entity.

Exit gate: owner migration access and restricted application access both work as intended.

## Phase 1.5 — Mutable business-state schema

Design and validate:

- `APP_USER`
- `ROLE`
- `USER_ROLE`
- `ACCOUNT`
- `BENEFICIARY`
- `RISK_POLICY`
- `RISK_POLICY_BAND`
- `PROTECTION_POLICY`
- `PAYMENT_TRANSACTION`
- `TRANSACTION_RISK_FACTOR`
- `OTP_CHALLENGE`
- `RISK_REVIEW`
- `IDEMPOTENCY_RECORD`
- `NOTIFICATION`
- `AUDIT_LOG`

Important requirements:

- Monetary columns use `NUMBER(18, 2)`.
- Payment amounts must be greater than zero.
- Account and transaction rows use optimistic-lock version columns.
- `risk_score` is nullable.
- Risk tier, policy version, explanation, and evaluation time are stored.
- Device and location columns do not exist in V1.
- OTP values are never stored in plaintext.
- Audit records are append-only from application behaviour.
- Foreign keys and uniqueness constraints prevent orphaned or

duplicate business records.

## Phase 1.6 — Ledger and exception schema

Create:

- LEDGER\_ENTRY
- TRANSACTION\_EXCEPTION

Ledger rules:

- Every completed settlement produces a balanced debit/credit pair.
- The prototype credits an internal SafePay clearing account rather than pretending to complete an external bank transfer.
- Ledger rows are append-only.
- A uniqueness key such as (source\_system, posting\_group\_key, posting\_line\_no) prevents duplicate posting while allowing both sides of a pair.
- Payment state change, balance update, ledger posting, audit record, and notification outbox work occur in one transaction boundary where required.

Exit gate: an invalid or unbalanced settlement cannot be accepted by the service design.

## Phase 1.7 — V1 policy seed data

Seed realistic but fictional records:

- Roles: CUSTOMER, RISK\_OFFICER, SYSTEM\_ADMIN, AUDITOR.
- Sample customers and one Risk Officer.
- Customer accounts with sufficient and insufficient balance cases.
- An internal clearing account.
- Active and inactive beneficiaries.
- V1 policy version such as AMOUNT\_ONLY\_V1.
- The four approved amount bands and protection durations.
- Example transactions covering each tier and terminal state.

Passwords must be BCrypt hashes generated through an approved

bootstrap process; plaintext credentials must not be committed to migration files.

Exit gate: migrations run from an empty schema, seed data loads, all constraints hold, and rerunning startup does not duplicate data.

---

## 4. Subplan 2 — Backend and REST API Development

---

Every phase follows the sequence:

```
database mapping → bean/entity → DAO → service  
contract → service implementation → DTOs → controller  
→ focused tests → API check → exit gate
```

### Phase 2.1 — Shared backend foundations

Implement first:

- Profile-based configuration.
- DTO and mapping conventions.
- Jakarta Validation.
- RFC 7807 `ProblemDetail` error responses.
- Global exception handling.
- UTC timestamp serialization.
- Correlation IDs.
- Constructor injection.
- Transaction boundaries.
- Security response handlers.
- Health endpoint.

Exit gate: malformed requests return consistent errors without leaking stack traces or internal SQL details.

### Phase 2.2 — User, roles, authentication, and

# authorization

Implement:

- `User`, `Role`, and `RoleName`.
- `UserDao` and `RoleDao`.
- `UserService` and implementation.
- `AuthService` and implementation.
- JWT creation and validation.
- Authentication filter.
- BCrypt password handling.
- Login throttling and account-lock behaviour.
- Role-based method and route protection.

APIs:

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/refresh`
- `POST /api/v1/auth/logout`

Security design:

- Short-lived access token held in frontend memory.
- Rotated refresh token through an `HttpOnly`, `Secure`, appropriately scoped cookie.
- No password, OTP, refresh token, or secret in logs.
- Production registration cannot assign privileged roles.

Exit gate: registration, login, refresh, logout, invalid credentials, expired tokens, locked accounts, and forbidden-role access work correctly.

## Phase 2.3 — Account module

Implement:

- `Account`
- `AccountDao`
- `AccountService` and implementation
- Account DTOs

- `AccountController`

APIs:

- `GET /api/v1/accounts`
- `GET /api/v1/accounts/{accountId}`
- `GET /api/v1/accounts/{accountId}/balance`

Rules:

- Customer ownership derives from the authenticated principal.
- A caller cannot retrieve another customer's account.
- Balances are never accepted from client-submitted transaction payloads.
- Balance-changing operations use database transaction control and locking.

Exit gate: ownership, not-found handling, inactive-account handling, and balance display are tested.

## Phase 2.4 — Beneficiary module

Implement:

- `Beneficiary`
- `BeneficiaryDao`
- `BeneficiaryService` and implementation
- Request/response DTOs
- `BeneficiaryController`

APIs:

- `POST /api/v1/beneficiaries`
- `GET /api/v1/beneficiaries`
- `GET /api/v1/beneficiaries/{beneficiaryId}`
- `PATCH /api/v1/beneficiaries/{beneficiaryId}/status`

Rules:

- Beneficiaries belong to a customer.
- Duplicate customer/account combinations are rejected.
- Disabled beneficiaries cannot receive new payments.
- Beneficiary age or novelty does not affect V1 risk.

Exit gate: create, retrieve, disable, duplicate, invalid account, ownership, and authorization scenarios pass.

## Phase 2.5 — Versioned V1 risk policy and engine

Implement:

- `RiskTier`
- `RiskPolicy`
- `RiskPolicyBand`
- `ProtectionPolicy`
- Corresponding DAOs and services
- `AmountRiskEngine`

Risk evaluation accepts a payment amount and returns:

- `riskTier`
- nullable `riskScore`
- protection duration
- matched rule or band
- policy/model version
- human-readable explanation
- evaluation timestamp

Boundary tests must use `BigDecimal` and cover:

- 0, negative values, and more than two decimal places
- 0.01
- 5000.00 and 5000.01
- 25000.00 and 25000.01
- 200000.00 and 200000.01
- extreme supported values
- null input

V2 extension: a future `RiskEngine` contract can introduce a weighted

implementation without changing transaction orchestration. Device/location and other signals would enter as new, separately approved feature modules.

Exit gate: all boundary tests pass and no controller duplicates risk-band logic.

## Phase 2.6 — Transaction creation and state engine

Implement:

- TransactionDb
- TransactionState
- TransactionRiskFactor
- TransactionDao
- TransactionRiskFactorDao
- TransactionStateService
- TransactionService
- DTOs
- TransactionController

APIs:

- POST /api/v1/transactions
- POST /api/v1/transactions/{transactionId}/authorize
- GET /api/v1/transactions
- GET /api/v1/transactions/{transactionId}
- GET /api/v1/transactions/{transactionId}/risk-explanation
- POST /api/v1/transactions/{transactionId}/cancel

State flow:

```
CREATED → AUTHORIZED → RISK_ASSESSED
```

LOW:

```
RISK_ASSESSED → RELEASED → SETTLED
```

MEDIUM/HIGH:

RISK\_ASSESSED → PROTECTED

PROTECTED → CANCELLED

PROTECTED → RELEASED → SETTLED

VERY\_HIGH:

RISK\_ASSESSED → VERIFICATION\_REQUIRED

VERIFICATION\_REQUIRED → PENDING\_RISK\_REVIEW

PENDING\_RISK\_REVIEW → RELEASED → SETTLED

PENDING\_RISK\_REVIEW → CANCELLED

PENDING\_RISK\_REVIEW → VERIFICATION\_REQUIRED

Technical failure:

eligible non-terminal state → FAILED

Rules:

- Clients never submit or select their risk tier.
- Every transition is validated centrally.
- Illegal transitions return a conflict response.
- Repeated requests cannot settle twice.
- Customers may cancel only before release.
- The protection deadline is calculated by the server.

Exit gate: every allowed and forbidden transition is tested.

## Phase 2.7 — Idempotency and request safety

Implement:

- `IdempotencyRecord`
- `IdempotencyRecordDao`
- `IdempotencyService`

All state-changing payment, OTP, and Risk Officer operations require `Idempotency-Key`.

Rules:

- Same key and same request returns the prior logical result.
- Same key with a different request payload returns a conflict.
- Concurrent duplicate requests produce one state change.
- Idempotency records retain response identity and request fingerprint.

Exit gate: retrying transaction creation, authorization, cancellation, approval, rejection, and settlement cannot duplicate effects.

## Phase 2.8 — Protection scheduler and cancellation races

Implement `ProtectedTransactionScheduler`.

Rules:

- Query using the database/server time for eligible `PROTECTED` rows whose deadline has expired.
- Release only rows still in `PROTECTED`.
- Use optimistic locking or conditional updates.
- Process in bounded batches.
- Recover expired rows after application restart.
- A cancellation and scheduler release racing against each other must have exactly one winner.

Exit gate: restart recovery, multiple application instances, expiry boundaries, cancel-at-expiry, and duplicate scheduler execution pass.

## Phase 2.9 — Ledger settlement and exception recording

Implement:

- `LedgerEntry`
- `LedgerEntryType`
- `TransactionExceptionLog`
- Ledger and exception DAOs
- `LedgerService`

Settlement transaction:

1. Lock or conditionally validate the payment and source account.
2. Reconfirm the expected payment state.
3. Verify sufficient available balance.
4. Apply source-account debit.
5. Create customer debit ledger line.
6. Create clearing-account credit ledger line.
7. Validate equal debit and credit totals.
8. Mark the payment `SETTLED`.
9. Append audit and notification records.
10. Commit atomically.

On failure:

- Roll back financial changes.
- Record an appropriate transaction exception through a safe separate path.
- Avoid partially settled states.

Exit gate: insufficient funds, duplicate settlement, database failure, and concurrent settlement tests prove atomicity.

## Phase 2.10 — OTP verification

Implement:

- `OtpChallenge`
- `OtpChallengeDao`
- `OtpService`
- `VerificationController`

APIs:

- `POST /api/v1/transactions/{transactionId}/otp`
- `POST /api/v1/transactions/{transactionId}/otp/verify`
- `POST /api/v1/transactions/{transactionId}/otp/resend`

Rules:

- OTP required only for eligible VERY\_HIGH payments.
- Store only a secure OTP hash.
- Enforce expiry, attempt limit, resend cooldown, and rate limiting.
- Invalidate previous OTP when a replacement is issued.
- Successful verification is single-use.
- Prototype delivery may use a development-only provider, clearly separated from production configuration.

Exit gate: correct, incorrect, expired, reused, brute-force, resend, and unauthorized OTP paths pass.

## Phase 2.11 — Risk Officer review

Implement:

- RiskReview
- RiskReviewStatus
- RiskReviewDao
- RiskReviewService
- RiskReviewController

APIs:

- GET /api/v1/admin/risk-reviews
- GET /api/v1/admin/risk-reviews/{reviewId}
- POST /api/v1/admin/risk-reviews/{reviewId}/approve
- POST /api/v1/admin/risk-reviews/{reviewId}/reject
- POST /api/v1/admin/risk-reviews/{reviewId}/request-verification
- POST /api/v1/admin/risk-reviews/{reviewId}/notes

Rules:

- Only RISK\_OFFICER or explicitly approved administrators can act.
- The reviewer cannot approve a non-pending review.
- Decision reason, reviewer, timestamp, prior state, and resulting state are audited.
- Two officers racing to decide must produce one final decision.

- Review status remains in `RISK_REVIEW`; unnecessary payment states are not introduced.

Exit gate: authorization, approve/reject/reverify, duplicate decision, concurrent decision, and audit completeness pass.

## Phase 2.12 — Audit, notifications, and WebSocket updates

Implement:

- `AuditLog`
- `Notification`
- DAOs and services
- Controllers
- `WebSocketConfig`

APIs:

- `GET /api/v1/audit-logs`
- `GET /api/v1/transactions/{transactionId}/audit`
- `GET /api/v1/notifications`
- `PATCH /api/v1/notifications/{notificationId}/read`

Real-time destinations communicate transaction-status changes, not authoritative financial decisions.

Rules:

- REST remains the source of truth after reconnect.
- Topic/user-destination authorization prevents cross-customer data access.
- Audit records contain actor, action, entity, prior/new state, correlation ID, and timestamp.
- Sensitive values are masked.

Exit gate: subscriptions are authorized, missed events reconcile through REST, and no customer receives another customer's event.

---

## 5. Subplan 3 — Complete Backend Verification and API Testing

---

### Phase 3.1 — Compilation and structural verification

Run:

- Clean Maven compilation.
- Unit tests.
- Dependency convergence checks.
- Configuration-property validation.
- Java 21 compatibility checks.
- Package and import verification.
- Startup under local and test profiles.

Exit gate: zero compilation errors, startup failures, unresolved imports, or ignored failing tests.

### Phase 3.2 — Oracle/JPA integration verification

Verify against Oracle, not H2:

- All Flyway migrations from an empty schema.
- Hibernate schema validation.
- Sequence allocation.
- Column precision and enum mapping.
- Foreign keys and unique constraints.
- Optimistic locking.
- Timestamp handling.
- Runtime-user privileges.
- Migration repeatability and upgrade path.

Exit gate: application startup and repository integration tests succeed against the actual target database.

### Phase 3.3 — Unit and service testing

Cover:

- Risk boundaries.
- State transition legality.
- Account ownership.
- Beneficiary validation.
- JWT validation.
- OTP policy.
- Review decisions.
- Ledger balancing.
- Idempotency.
- Exception translation.
- Audit generation.

Exit gate: all business branches have deterministic tests with no reliance on browser timing.

## **Phase 3.4 — Postman/API progression**

Build and run a Postman collection in the same order as development:

1. Register and authenticate.
2. Obtain and refresh credentials.
3. Retrieve accounts.
4. Create and manage beneficiaries.
5. Create and authorize LOW transaction.
6. Exercise MEDIUM cancellation and automatic release.
7. Exercise HIGH cancellation and automatic release.
8. Exercise VERY\_HIGH OTP and approval.
9. Exercise VERY\_HIGH rejection.
10. Exercise verification request.
11. Retrieve audit history and notifications.
12. Repeat mutation requests using identical and conflicting idempotency keys.

Validate status codes, response bodies, headers, authorization, ownership, and database effects.

Exit gate: the complete collection runs without unexplained failure.

## Phase 3.5 — Concurrency and recovery testing

Test:

- Cancel versus expiry release.
- Two scheduler workers selecting the same payment.
- Two Risk Officers deciding the same review.
- Two settlement attempts.
- Duplicate transaction creation.
- Balance update collisions.
- Application restart during protection.
- WebSocket disconnect/reconnect.
- Database interruption during ledger posting.

Exit gate: no double settlement, negative balance caused by races, split ledger, duplicate decision, or illegal terminal-state change.

## Phase 3.6 — Security verification

Test:

- Missing, malformed, expired, and forged JWTs.
- Refresh-token rotation and replay.
- Customer access to another customer's resources.
- Customer access to Risk Officer endpoints.
- Privilege escalation during registration.
- SQL injection and invalid identifiers.
- Oversized payloads.
- OTP brute-force and resend abuse.
- CORS, CSP, secure-cookie, and security-header behaviour.
- Log and error-message data leakage.
- WebSocket destination authorization.

Exit gate: all critical security failures are corrected or explicitly accepted and documented by the user.

## Phase 3.7 — Backend completion gate

Backend is complete only when:

- Build and all required tests pass.
  - Postman regression passes.
  - Oracle schema validates.
  - All amount bands and state flows pass.
  - Ledger entries balance.
  - Security checks pass.
  - Carry-forward items are either resolved or explicitly tied to frontend/deployment work.
  - No feature is marked complete merely because its dependent test has been deferred.
- 

## 6. Subplan 4 — Oracle JET PWA Frontend

---

### Phase 4.1 — Frontend prerequisites and scaffold

Install or validate:

- Node.js 24 LTS.
- npm-compatible tooling.
- Oracle JET 19 tooling.
- TypeScript support.

Create an Oracle JET MVVM application using Knockout and standard/ Core Pack JET components. Do not introduce React, Vue, or Vite component frameworks.

Exit gate: the clean OJET application builds, serves locally, and communicates with the backend health endpoint.

### Phase 4.2 — Responsive design system and application shell

Implement:

- Oracle Redwood visual theme.
- Responsive navigation.
- Customer and Risk Officer layouts.
- Accessible form, dialog, status, table, and card patterns.
- Loading, empty, offline, and error states.
- Protected route handling.
- Session-expiry handling.

Validate widths including:

- 360 px
- 390 px
- 600 px
- 768 px
- 1024 px
- 1440 px

Mobile uses cards, drawers, and reachable bottom actions. Desktop uses full tables, panels, and wider investigation views. Interactive targets are at least 44 px.

Exit gate: no overflow, clipped action, inaccessible dialog, or unusable navigation at approved widths.

## **Phase 4.3 — Authentication and account experience**

Implement:

- Registration.
- Login.
- Logout.
- Session refresh.
- Account dashboard.
- Balance and account details.
- Role-aware navigation.

Access tokens remain in memory. Sensitive tokens and payment data are not stored in local storage or PWA caches.

Exit gate: login and refresh work on desktop and phone layouts without

exposing credentials.

## **Phase 4.4 — Beneficiary experience**

Implement:

- Beneficiary list.
- Beneficiary creation.
- Beneficiary details.
- Enable/disable action.
- Validation and backend error presentation.

Exit gate: the frontend respects ownership and disabled-beneficiary behaviour.

## **Phase 4.5 — Customer payment journey**

Implement:

1. Select account.
2. Select or review beneficiary.
3. Enter amount.
4. Review and authorize payment.
5. Render the server-returned risk tier and explanation.
6. Show the correct outcome:
  - LOW: settlement confirmation.
  - MEDIUM: 10-second protection window.
  - HIGH: 60-second protection window and warning.
  - VERY\_HIGH: verification and Risk Officer hold.

Rules:

- The client never calculates the authoritative risk tier.
- Countdown display is derived from the server deadline.
- Undo uses a protected, idempotent backend call.
- Final status is fetched from REST after reconnection or timer completion.
- Double-submit controls complement, but never replace, server idempotency.

Exit gate: all four journeys work across approved viewport sizes.

## **Phase 4.6 — OTP and held-payment experience**

Implement:

- Masked OTP destination.
- OTP entry.
- Expiry and resend state.
- Failed-attempt feedback.
- Pending Risk Officer review screen.
- Customer cancellation while still eligible.
- Approval, rejection, or re-verification status.

Exit gate: refreshing or reopening the page reconstructs the correct state from the backend.

## **Phase 4.7 — Risk Officer dashboard**

Implement:

- Pending-review queue.
- Review details.
- Customer, beneficiary, amount, risk tier, and explanation.
- OTP-verification status.
- Audit timeline.
- Approve, reject, request verification, and add-note actions.
- Confirmation dialogs and mandatory reasons where required.

V1 must not display unsupported device/location signals or pretend that historical fraud-pattern analysis exists.

Exit gate: role protection and concurrent-decision conflicts are correctly displayed.

## **Phase 4.8 — Notifications and live updates**

Implement:

- WebSocket/STOMP connection.
- User-specific status events.
- Risk Officer queue refresh.
- Notification centre.
- REST resynchronization after reconnect.
- Duplicate-event handling.

Exit gate: losing the socket never changes financial correctness or leaves the UI permanently stale.

## **Phase 4.9 — PWA and offline safety**

Implement:

- Web app manifest.
- Installable icons and metadata.
- Service worker.
- Static application-shell caching.
- Offline notification.
- Safe update behaviour.

Never cache:

- Authentication responses.
- Tokens.
- Account balances.
- Beneficiaries.
- Transactions.
- OTP data.
- Risk-review details.
- Audit data.

Financial mutations are disabled offline and are not silently queued.

Exit gate: installability checks pass, the static shell opens safely offline, and sensitive API data is absent from caches.

## **Phase 4.10 — Frontend quality gate**

Verify:

- Responsive behaviour.
- Keyboard navigation.
- Focus management.
- Screen-reader labels.
- Colour contrast.
- Reduced-motion behaviour.
- Browser refresh and deep links.
- Session expiry.
- Slow network and backend errors.
- Mobile installation.
- Full customer and Risk Officer end-to-end flows.

Exit gate: frontend build and all critical journeys pass without blocking visual, accessibility, or state-consistency defects.

---

## 7. Subplan 5 — Deployment and Server Settings

---

### Phase 5.1 — Production configuration

Prepare:

- `application-prod.yml` without embedded secrets.
- Environment variables or secret-store mappings.
- Oracle connection-pool settings.
- JWT key management.
- OTP provider configuration.
- Allowed origins.
- Secure cookie settings.
- Rate limits.
- Actuator exposure restrictions.
- Structured logging and correlation IDs.
- UTC server and database time policy.

Exit gate: no development password, sample secret, or privileged database credential exists in deployable artifacts.

## Phase 5.2 — Frontend production build and hosting

Build the Oracle JET application and serve it through the selected web server or Spring Boot static hosting arrangement.

Preferred prototype layout:

```
Browser/PWA
  → HTTPS same-origin host
    → Oracle JET static application
      → /api/v1/**
      → /ws/**
  → Spring Boot application
  → Oracle Database
```

Same-origin deployment minimizes CORS and cookie complexity. Client-side routes must fall back to the frontend entry page without capturing API or WebSocket paths.

Exit gate: browser refresh, deep links, REST calls, cookies, and WebSockets all work through the deployment endpoint.

## Phase 5.3 — Database deployment

Deployment order:

1. Back up the target schema.
2. Validate database version and free space.
3. Run Flyway migrations as schema owner.
4. Validate objects and grants.
5. Start the application as restricted runtime user.
6. Verify health.
7. Run smoke tests.
8. Retain a documented recovery approach for failed deployment.

Exit gate: migrations succeed from both a clean database and the last supported schema version.

## Phase 5.4 — HTTPS and server hardening

Configure:

- TLS.
- HTTP-to-HTTPS redirect.
- HSTS after HTTPS verification.
- Content Security Policy.
- `X-Content-Type-Options`.
- Referrer policy.
- Frame protection.
- Request body limits.
- Connection and read timeouts.
- Secure proxy headers.
- Restricted management endpoints.
- Log rotation.
- Non-root application process where supported.

Exit gate: production security headers, cookies, redirects, and proxy behaviour pass verification.

## Phase 5.5 — Operational readiness

Provide:

- Startup and shutdown instructions.
- Health and readiness checks.
- Database-connectivity checks.
- Scheduler monitoring.
- Failed-transaction and reconciliation queries.
- Audit-log access procedure.
- Backup and restore procedure.
- Log locations and rotation.
- Common incident troubleshooting steps.
- Demo-user and sample-data reset process.

- Release checklist.

Exit gate: the application survives restart, recovers eligible protected transactions, preserves held reviews, and does not duplicate settlement.

## Phase 5.6 — Final acceptance

Run the complete release scenario:

- Customer login.
- Account and beneficiary verification.
- LOW instant payment.
- MEDIUM protected payment with cancellation.
- MEDIUM automatic release.
- HIGH protected payment with cancellation.
- HIGH automatic release.
- VERY\_HIGH OTP and approval.
- VERY\_HIGH rejection.
- VERY\_HIGH re-verification.
- Audit and notification verification.
- Duplicate-request replay.
- Unauthorized access attempts.
- Restart recovery.
- Responsive desktop and mobile inspection.
- PWA installation and offline-safety inspection.

The prototype is accepted only when all mandatory tests pass and no unresolved issue can compromise financial correctness, security, auditability, or the approved customer journey.

---

## 8. Public Interface and Data Contract Rules

---

These remain unchanged throughout implementation:

- Base REST path: `/api/v1`.
- JSON API DTOs only; JPA entities are never returned directly.

- Monetary values use decimal-safe representations.
  - Timestamps use ISO-8601 UTC.
  - The authenticated principal determines customer ownership.
  - State-changing transaction, OTP, and review requests require `Idempotency-Key`.
  - Error responses use RFC 7807-compatible `ProblemDetail`.
  - WebSocket events are advisory; REST and the database remain authoritative.
  - Risk results contain tier, nullable score, policy version, explanation, and relevant deadline/status.
  - Sensitive fields are never returned or logged.
- 

## 9. Explicit V2 Extension Boundary

---

V2 may add:

- Weighted multi-factor scores.
- New-beneficiary signals.
- Transaction velocity.
- Amount-anomaly analysis.
- Device trust.
- Location signals.
- Configurable tier thresholds.
- User safety preferences.
- Broader maker-checker workflows.
- Dispute management.
- Risk analytics views.
- Read-only investigation assistance.

The reusable boundary is a risk-engine contract whose V1 implementation is `AmountRiskEngine`. A V2 engine may consume a richer evaluation context and return the same stable risk-result contract. Transaction orchestration, state validation, ledger settlement, auditing, and API security must not depend on the internal scoring formula.

Device and location require a separately approved feature cycle covering consent, retention, schema, APIs, security, frontend capture,

and testing; they are not dormant V1 columns.

---

## 10. PDF Deliverable

---

No PDF or project file is generated during this planning turn.

After the user approves this complete plan and explicitly requests artifact generation in an execution-capable mode:

1. Convert this approved plan into a professionally formatted PDF.
2. Include the development charter, package structure, five subplans, state flow, API catalogue, security requirements, tests, gates, and V2 boundary.
3. Render and visually inspect every page.
4. Correct clipping, broken page breaks, unreadable tables, or inconsistent headings.
5. Save the final PDF inside the SafePay project only after explicit authorization.
6. Present the finished PDF as a downloadable local-file link.

## Assumptions and Defaults

---

- Java 21 LTS is mandatory across development and deployment.
- Spring Boot remains within the approved 3.x family.
- Oracle Database Free with `FREEPDB1` is the local database target.
- SQL Developer is the primary graphical database tool.
- Oracle JET 19 MVVM with Knockout and TypeScript is the frontend target.
- Backend and frontend are developed separately and deployed under one origin for the prototype.
- The user manually creates and edits implementation files using guided content.
- A phase may advance only after its exit gate passes; deferrals must name the exact future dependency.

- The supplied layered packages—beans, controller, repository, services, and excp—are authoritative for the V1 project structure.
- All earlier approved SafePay flows, APIs, ledger integrity measures, security controls, and PWA requirements remain preserved.